

Compte Rendu TP 5 : Traitement d'Image

Gabriel Mazet - L3 Informatique - Programmation Parallèle (OpenMP)

Présentation du Problème et Algorithmes

L'objectif de ce TP est de paralléliser un traitement d'image complet en C avec OpenMP. Le traitement se fait en deux étapes :

1. **La conversion en niveaux de gris** : conversion d'une image couleur au format PPM (3 octets par pixel, format P3 ASCII) vers une image en niveaux de gris au format PGM (1 octet par pixel, format P2 ASCII) selon la formule de luminance :

$$g = (299 * r + 587 * g + 114 * b) / 1000$$

2. **L'égalisation d'histogramme** : amélioration du contraste de l'image en niveaux de gris. Dans une image peu contrastée, les pixels se concentrent sur des valeurs proches. L'égalisation permet d'aplanir l'histogramme pour utiliser toute la dynamique disponible (de 0 à 255).

L'algorithme d'égalisation d'histogramme :

- **Histogramme H** : on compte pour chaque niveau de gris i (de 0 à 255) le nombre de pixels qui possèdent cette valeur (noté $H[i]$).
- **Histogramme cumulé C** : on calcule $C[i] = C[i-1] + H[i]$, qui représente le nombre de pixels dont la valeur est inférieure ou égale à i .
- **Transformation** : on remplace chaque pixel d'origine de valeur $P[i]$ par sa nouvelle valeur étirée :

$$P_{\text{nouveau}}[i] = (255 * C[P[i]]) / S$$

où S est le nombre total de pixels de l'image ($\text{largeur} * \text{hauteur}$).

Stratégie de Parallélisation OpenMP

Afin d'obtenir les meilleures performances possibles, le programme complet a été parallélisé. Les entrées/sorties (chargement PPM et écriture PGM) restent séquentielles car la lecture de fichiers ASCII n'est pas parallélisable de manière simple en OpenMP.

A. Conversion couleur vers niveaux de gris (`colorToGreyOmp`)

Chaque pixel de l'image couleur est traité de façon totalement indépendante. On utilise une simple directive `#pragma omp parallel for` sur la boucle linéaire de taille `width * height`.

B. Égalisation d'histogramme (`enhanceContrastOmp`)

Pour éviter d'enchaîner plusieurs régions parallèles (ce qui génère de l'overhead de création de threads), j'ai encapsulé toute la fonction dans une unique région parallèle `#pragma omp parallel` :

1. **Étape 1 : Histogrammes locaux (Évitement de data race)**. Si tous les threads incrémentaient directement l'histogramme global partagé H , il y aurait des conflits d'accès (data race). Utiliser une section `critical` à chaque pixel détruirait les performances. À la place, chaque thread crée son propre tableau d'histogramme local `localH` sur sa pile, et remplit cette copie de façon indépendante avec `#pragma omp for nowait`.

2. **Étape 2 : Fusion (Exclusion mutuelle).** Chaque thread ajoute son histogramme local à l'histogramme global `H` dans un bloc `#pragma omp critical`. Comme cette opération n'a lieu qu'une seule fois par thread, l'overhead est complètement négligeable.
3. **Étape 3 : Barrière et Histogramme cumulé.** On insère un `#pragma omp barrier` pour garantir que tous les threads ont fini de fusionner leurs données dans `H`. Ensuite, un seul thread calcul l'histogramme cumulé `C` de manière séquentielle via `#pragma omp single`.
4. **Étape 4 : Transformation finale.** Les threads se répartissent le calcul de la transformation des pixels de l'image avec un dernier `#pragma omp for`.

```
void enhanceContrastOmp(GreyImage *image, int numThreads) {
    int size = image->width * image->height;
    long long h[256] = {0};
    long long c[256] = {0};
    omp_set_num_threads(numThreads);

    #pragma omp parallel
    {
        long long localH[256] = {0};

        #pragma omp for nowait
        for (int i = 0; i < size; i++) {
            localH[image->pixels[i]]++;
        }

        #pragma omp critical
        {
            for (int k = 0; k < 256; k++) {
                h[k] += localH[k];
            }
        }

        #pragma omp barrier

        #pragma omp single
        {
            c[0] = h[0];
            for (int i = 1; i < 256; i++) {
                c[i] = c[i - 1] + h[i];
            }
        } // barrière implicite à la fin du single

        #pragma omp for
        for (int i = 0; i < size; i++) {
            image->pixels[i] = (unsigned char)((255 * (long long)c[image->pixels[i]]) / size);
        }
    }
}
```

Résultats et Mesures de Performances

Les mesures ont été effectuées sur les 3 images fournies, en prenant la moyenne sur **5 exécutions** par configuration. Les temps indiqués correspondent au temps de calcul pur (conversion en gris + égalisation), excluant les entrées/sorties sur disque.

A. Image 0 (299 x 168 px, ~50 kpx)

Configuration	Temps Gris (s)	Temps Contraste (s)	Temps Total (s)	Speedup Gris	Speedup Contraste	Speedup Total
Séquentiel	0.000077	0.000410	0.000487	1.00x	1.00x	1.00x

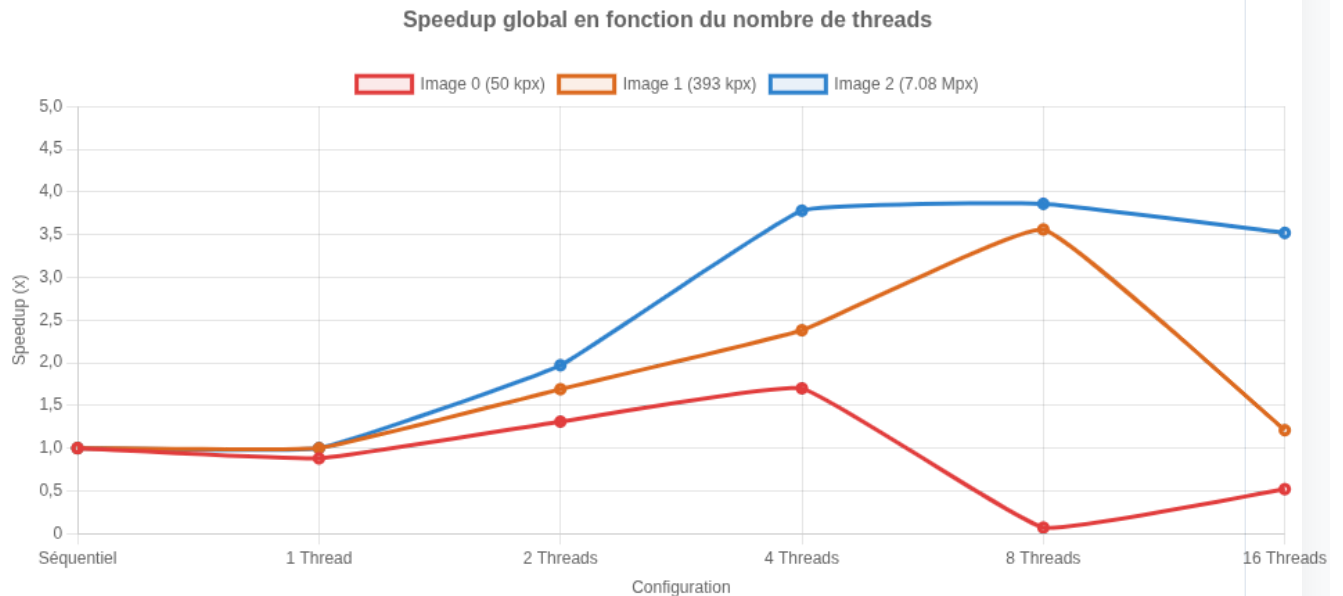
Configuration	Temps Gris (s)	Temps Contraste (s)	Temps Total (s)	Speedup Gris	Speedup Contraste	Speedup Total
1 Thread	0.000094	0.000459	0.000554	0.82x	0.89x	0.88x
2 Threads	0.000147	0.000224	0.000371	0.53x	1.83x	1.31x
4 Threads	0.000173	0.000114	0.000287	0.45x	3.60x	1.70x
8 Threads	0.001832	0.005261	0.007094	0.04x	0.08x	0.07x
16 Threads	0.000541	0.000387	0.000929	0.14x	1.06x	0.52x

B. Image 1 (768 x 512 px, ~393 kpx)

Configuration	Temps Gris (s)	Temps Contraste (s)	Temps Total (s)	Speedup Gris	Speedup Contraste	Speedup Total
Séquentiel	0.000606	0.003216	0.003822	1.00x	1.00x	1.00x
1 Thread	0.000627	0.003197	0.003825	0.97x	1.01x	1.00x
2 Threads	0.000464	0.001795	0.002259	1.31x	1.79x	1.69x
4 Threads	0.000431	0.001171	0.001603	1.41x	2.75x	2.38x
8 Threads	0.000413	0.000661	0.001074	1.47x	4.86x	3.56x
16 Threads	0.001118	0.002045	0.003162	0.54x	1.57x	1.21x

C. Image 2 (3072 x 2304 px, ~7.08 Mpx)

Configuration	Temps Gris (s)	Temps Contraste (s)	Temps Total (s)	Speedup Gris	Speedup Contraste	Speedup Total
Séquentiel	0.010880	0.057643	0.068523	1.00x	1.00x	1.00x
1 Thread	0.010915	0.057663	0.068578	1.00x	1.00x	1.00x
2 Threads	0.005776	0.029034	0.034809	1.88x	1.99x	1.97x
4 Threads	0.003330	0.014807	0.018136	3.27x	3.89x	3.78x
8 Threads	0.003886	0.013844	0.017730	2.80x	4.16x	3.86x
16 Threads	0.004678	0.014814	0.019492	2.33x	3.89x	3.52x



Analyse des performances

L'impact de la taille des données (Overhead OpenMP)

Pour la toute petite image (`image0`), on observe que le parallélisme n'est pas du tout efficace au-delà de 4 threads (le speedup s'effondre à 0.07x pour 8 threads). La raison est simple : le volume de calcul sur 50 000 pixels est extrêmement faible (~0.4 milliseconde). Le coût de création des threads OpenMP et les étapes de synchronisation (notamment les barrières) prennent plus de temps que le calcul lui-même. C'est l'**overhead** d'OpenMP.

Scaling et saturation (Image 2)

Pour la grande image (`image2` , 7.08 millions de pixels), le gain est presque parfaitement linéaire jusqu'à 4 threads (**speedup de 3.78x pour 4 threads**, soit une efficacité parallèle de 94.5%). Cependant, l'accélération sature à 8 threads (3.86x) et régresse avec 16 threads (3.52x).

Cette saturation s'explique par deux phénomènes :

- **Nombre de cœurs physiques** : La machine de test possède 4 cœurs physiques et 8 cœurs logiques (HyperThreading). Le gain d'un thread logique par rapport à un cœur physique est limité car ils partagent les mêmes unités d'exécution. Configurer 16 threads sur un processeur doté de seulement 8 threads logiques crée de l'ordonnancement inutile et dégrade le temps global.
- **Bande passante mémoire (Memory Bound)** : Les opérations de conversion et de transformation de pixels sont très légères en calcul (quelques additions et multiplications) mais manipulent de gros volumes de données. Le bus mémoire est vite saturé par la demande simultanée de tous les cœurs, empêchant une accélération linéaire au-delà des cœurs physiques.

Efficacité de l'Égalisation vs la Conversion en Gris

On remarque que l'égalisation d'histogramme obtient de meilleurs speedups individuels (jusqu'à 4.16x sur image2) que la conversion en niveaux de gris (maximum 3.27x). Cela provient du fait que la fonction d'égalisation effectue plus d'opérations locales et profite mieux de l'architecture de région parallèle unique où les variables locales restent au chaud dans les caches L1/L2 des différents processeurs.

Conclusion

Ce TP montre qu'OpenMP est une solution redoutablement efficace pour accélérer les traitements d'image à condition que la taille des données soit suffisante pour masquer l'overhead d'initialisation. Notre approche avec **région parallèle unique** et **réduction manuelle locale** a permis d'optimiser l'égalisation d'histogramme en évitant toute data race sans pour autant introduire de verrous coûteux au sein de la boucle de traitement. Les résultats obtenus sont cohérents avec les caractéristiques matérielles du processeur de test.